МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 11

Тема: Поведенческие паттерны: Стратегия, Шаблонный метод

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы поведенческих паттернов проектирования «Стратегия» (Strategy) и «Шаблонный метод» (Template Method) и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать два поведенческих паттерна из списка: «Стратегия», «Шаблонный метод».

В соответствии с заданием были выбраны паттерны «Стратегия» (Strategy) и «Шаблонный метод» (Template Method).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, а также демонстрирует работу поведенческих паттернов: стратегия позволяет переключать алгоритм хэширования, а шаблонный метод задаёт единый скелет операций над паролем.

1. Описание паттерна «Стратегия»

Паттерн «Стратегия» (Strategy) - это поведенческий паттерн проектирования, который позволяет менять алгоритм работы объекта во время выполнения, инкапсулируя каждый алгоритм в отдельный класс и делая их взаимозаменяемыми.

Данный паттерн применяется, когда у класса есть несколько вариантов поведения, и нужно выбирать один из них в зависимости от ситуации или настроек.

В разработанном приложении паттерн «Стратегия» используется для выбора алгоритма хэширования пароля. В программе определены роли:

- HashingStrategy - общий интерфейс стратегии с методами hash(sequence) и get_name();

- SHA256Strategy - хэширование через SHA-256;

- MD5Strategy - хэширование через MD5;

- LengthStrategy - упрощённая стратегия вида len:N:sum:M для наглядной демонстрации.

PasswordManager хранит ссылку на текущую стратегию и делегирует ей вычисление хэша:

self.password_manager.set_strategy(new_strategy)

Пользователь может переключать стратегию кнопкой «Сменить стратегию», и все последующие операции используют новую стратегию.

Клиентский код (PasswordManager, PasswordOperation) не меняется - он работает через общий интерфейс HashingStrategy.

2. Описание паттерна «Шаблонный метод»

Паттерн «Шаблонный метод» (Template Method) - это поведенческий паттерн проектирования, который определяет скелет алгоритма в базовом классе, оставляя реализацию отдельных шагов подклассам. Подклассы могут переопределять шаги, но не могут менять структуру алгоритма.

Данный паттерн применяется, когда несколько классов выполняют похожие операции с одинаковым порядком шагов, но разной реализацией отдельных шагов.

В разработанном приложении паттерн «Шаблонный метод» используется для операций над паролем. В программе определены роли:

- PasswordOperation - абстрактный класс с методом execute(sequence), который задаёт фиксированный порядок шагов:
- validate(sequence) - проверка последовательности;
- prepare(sequence) - подготовка (по умолчанию - копирование);
- do_hash(sequence) - вычисление хэша (абстрактный);
- save(sequence, hash_value) - сохранение или сравнение (абстрактный);
- after_save(sequence, hash_value) - хук после сохранения (по умолчанию ничего);
- success_message() - сообщение при успехе (абстрактный);
- CreatePasswordOperation - операция создания пароля;
- VerifyPasswordOperation - операция проверки пароля.

Метод execute реализован один раз в базовом классе и не переопределяется. Подклассы переопределяют только абстрактные шаги. Это гарантирует, что все операции над паролем выполняются в едином порядке.

3. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter.

Основное окно содержит:

- сетку 3×3 из девяти скруглённых ячеек;
- отображение текущей последовательности;
- селектор стратегии хэширования («SHA-256» / «MD5» / «Length+Sum»);
- предпросмотр текущего хэша выбранной последовательности;
- основную кнопку («Войти» / «Сохранить пароль»);
- кнопку «Очистить»;
- переключатель режимов «Вход» / «Создать пароль»;
- кнопку «Сбросить сохранённый пароль».

К паролю предъявляются требования:

- длина от 4 до 9 ячеек;
- отсутствие повторов;
- ячейки в диапазоне 1–9.

Файл graphical_password.json хранит хэш пароля и название стратегии, которая использовалась при его создании.

4. Диаграмма классов

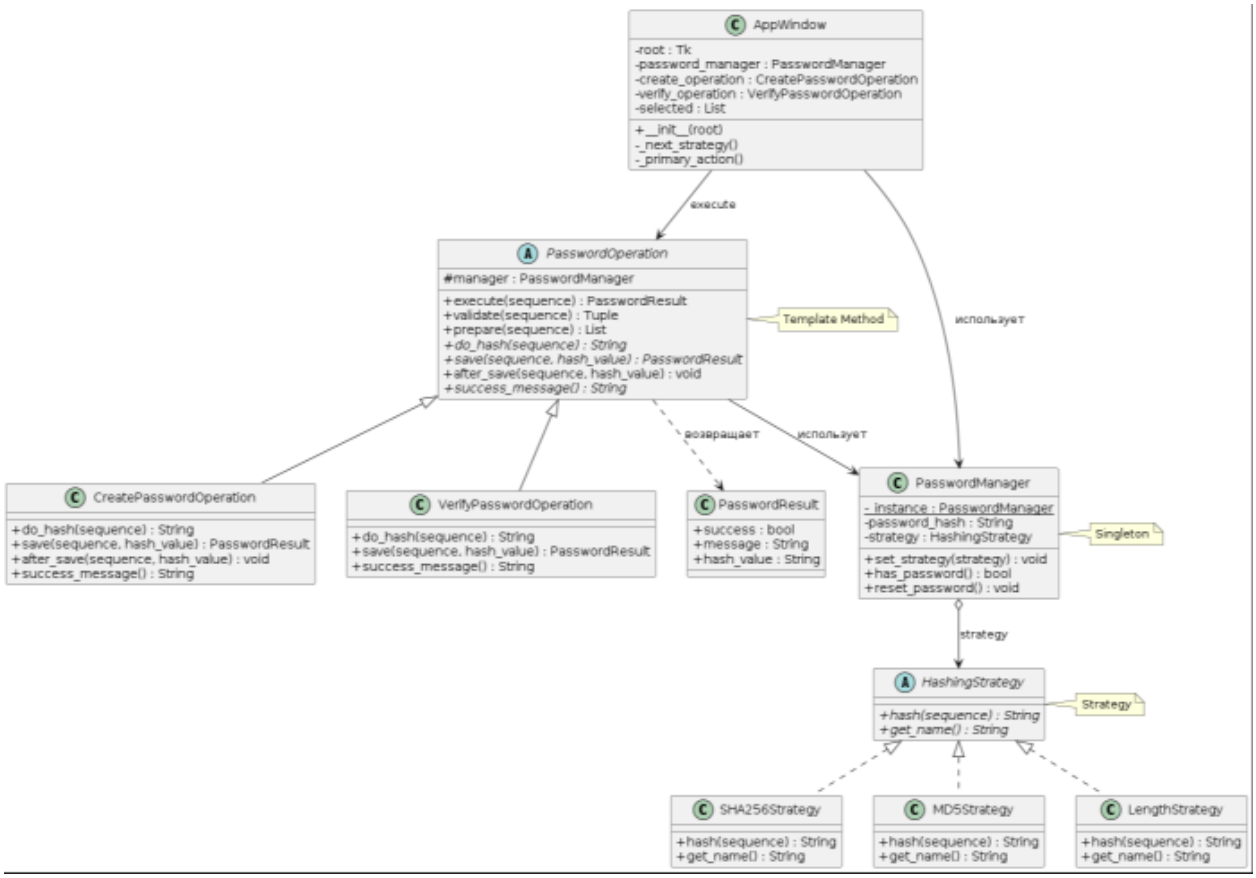


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

5. Реализация паттерна «Стратегия»

Интерфейс стратегии:

```
class HashingStrategy(ABC):
    @abstractmethod
    def hash(self, sequence):
        pass

    @abstractmethod
    def get_name(self):
        pass
```

Конкретные стратегии:

```
class SHA256Strategy(HashingStrategy):
    def hash(self, sequence):
        data = "".join(map(str, sequence))
```

```
        return hashlib.sha256(data.encode('utf-8')).hexdigest()

    def get_name(self):
        return "SHA-256"

class MD5Strategy(HashingStrategy):
    def hash(self, sequence):
        data = "".join(map(str, sequence))
        return hashlib.md5(data.encode('utf-8')).hexdigest()

    def get_name(self):
        return "MD5"

class LengthStrategy(HashingStrategy):
    def hash(self, sequence):
        return f"len:{len(sequence)}:sum:{sum(sequence)}"

    def get_name(self):
        return "Length+Sum"
```

PasswordManager хранит и использует текущую стратегию:

```
def set_strategy(self, strategy):
    self.strategy = strategy
```

Смена стратегии в UI:

```
def _next_strategy(self):
    self.strategy_index = (self.strategy_index + 1) % len(self.strategies)
    new_strategy = self.strategies[self.strategy_index]
    self.password_manager.set_strategy(new_strategy)
```

6. Реализация паттерна «Шаблонный метод»

Базовый класс со скелетом алгоритма:

```
class PasswordOperation(ABC):
```

```
def __init__(self, manager):
    self.manager = manager

def execute(self, sequence):
    ok, msg = self.validate(sequence)
    if not ok:
        return PasswordResult(False, msg)

    prepared = self.prepare(sequence)
    hash_value = self.do_hash(prepared)

    save_result = self.save(sequence, hash_value)
    if not save_result.success:
        return save_result

    self.after_save(sequence, hash_value)
    return PasswordResult(True, self.success_message(), hash_value)

def validate(self, sequence):
    return validate_sequence(sequence)

def prepare(self, sequence):
    return list(sequence)

@abstractmethod
def do_hash(self, sequence):
    pass

@abstractmethod
def save(self, sequence, hash_value):
```

```
        pass

    def after_save(self, sequence, hash_value):
        pass

    @abstractmethod
    def success_message(self):
        pass
```

Конкретная операция создания пароля:

```
class CreatePasswordOperation(PasswordOperation):
    def do_hash(self, sequence):
        return self.manager.strategy.hash(sequence)

    def save(self, sequence, hash_value):
        self.manager.password_hash = hash_value
        self.manager.save_to_file()
        return PasswordResult(True, "")

    def after_save(self, sequence, hash_value):
        self.manager.notify_strategy_used(self.manager.strategy.get_name())

    def success_message(self):
        return "Графический пароль успешно создан."
```

Конкретная операция проверки пароля:

```
class VerifyPasswordOperation(PasswordOperation):
    def do_hash(self, sequence):
        return self.manager.strategy.hash(sequence)

    def save(self, sequence, hash_value):
        if not self.manager.has_password():
```

```
        return PasswordResult(False, "Пароль не установлен.")
    if hash_value == self.manager.password_hash:
        return PasswordResult(True, "")
    return PasswordResult(False, "Пароль неверный. Доступ запрещён.")

def success_message(self):
    return "Пароль верный. Доступ разрешён."
```

7. Результаты работы программы

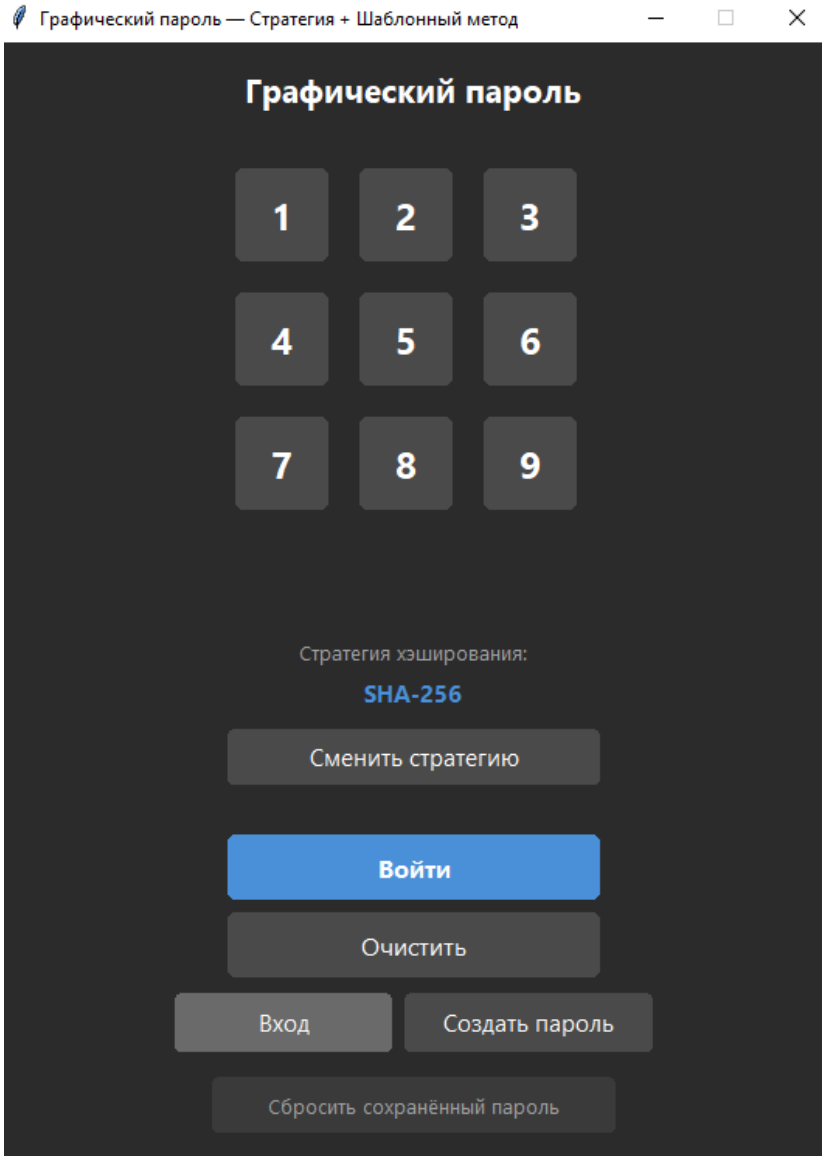


Рисунок 2 - Главное окно приложения (стратегия SHA-256)

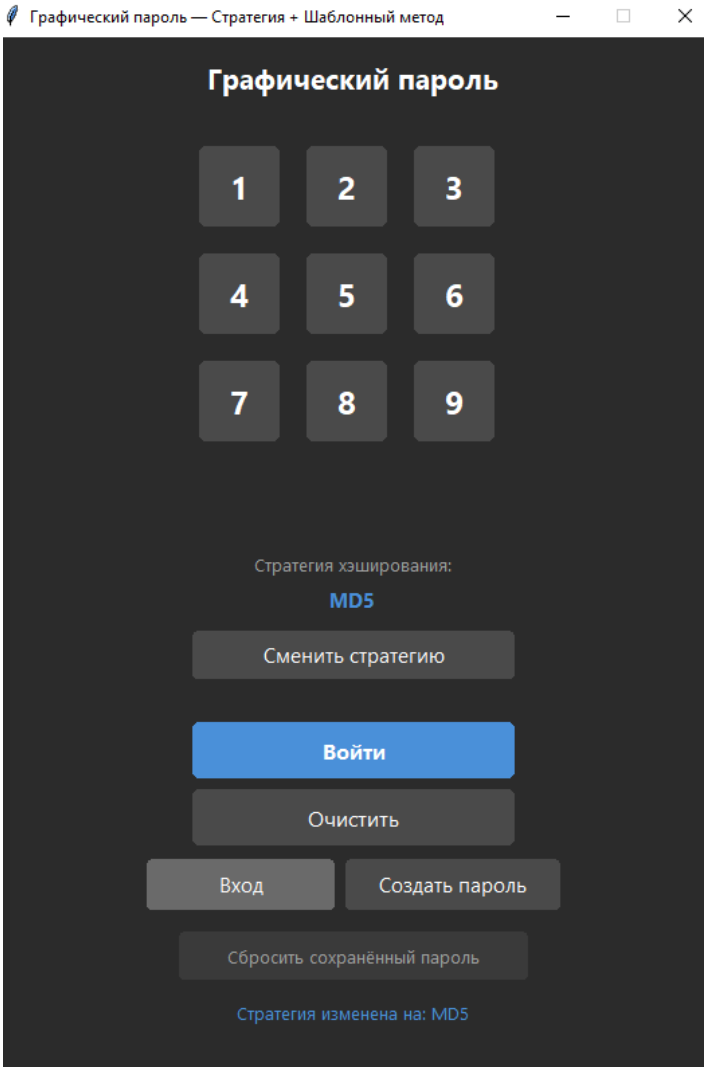


Рисунок 3 - Смена стратегии на MD5

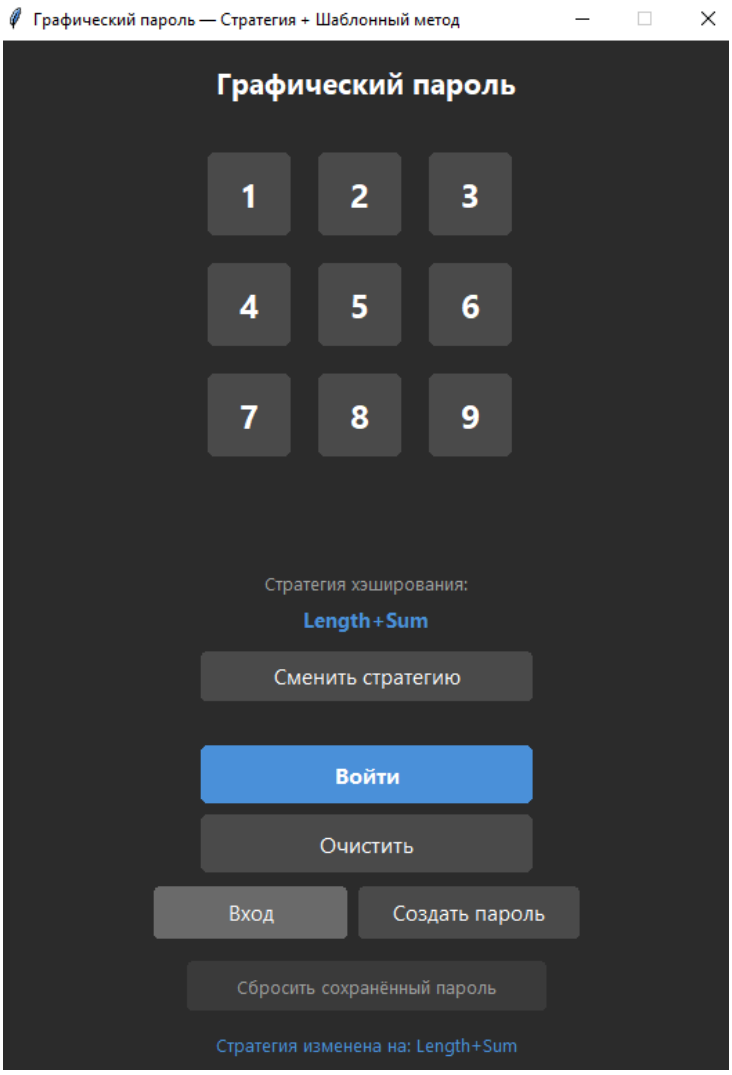


Рисунок 4 - Смена стратегии на Length+Sum

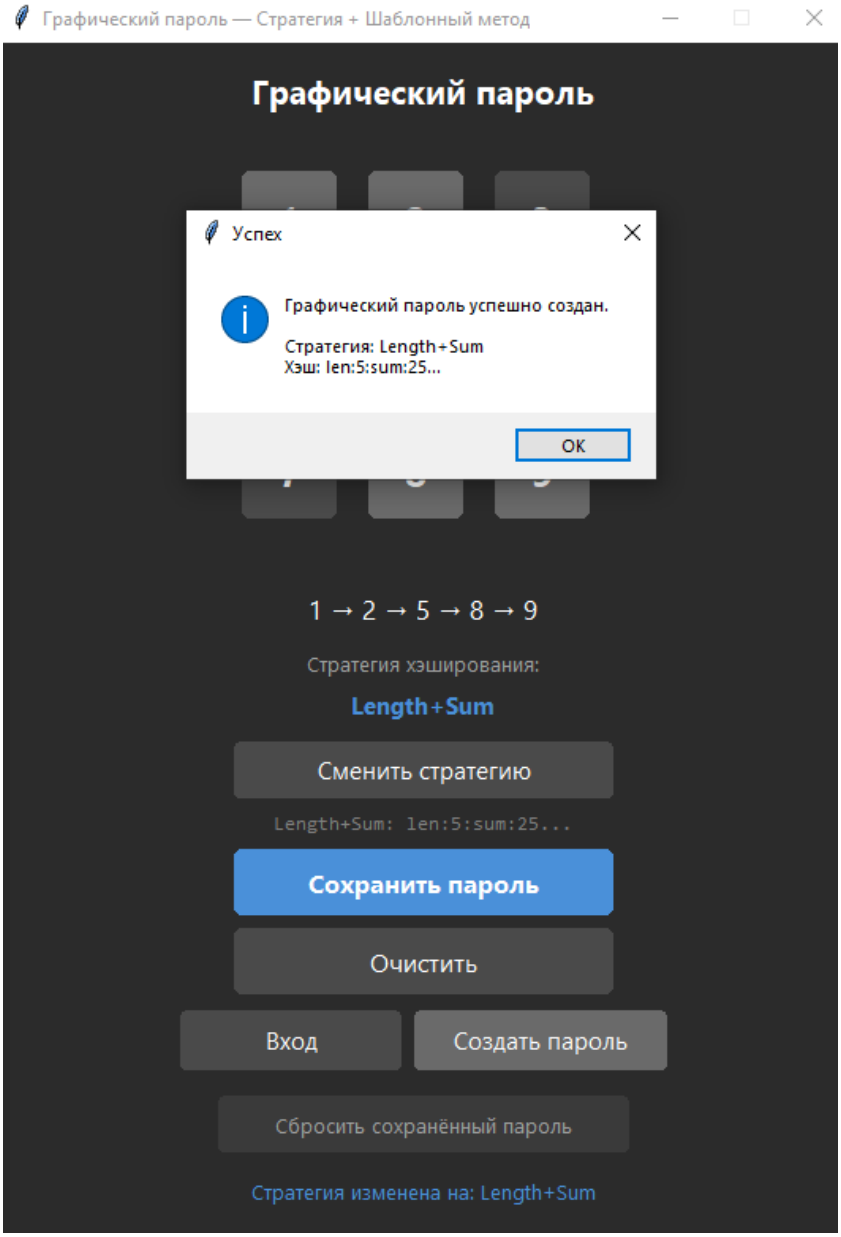


Рисунок 5 - Сохранение пароля с выбранной стратегией

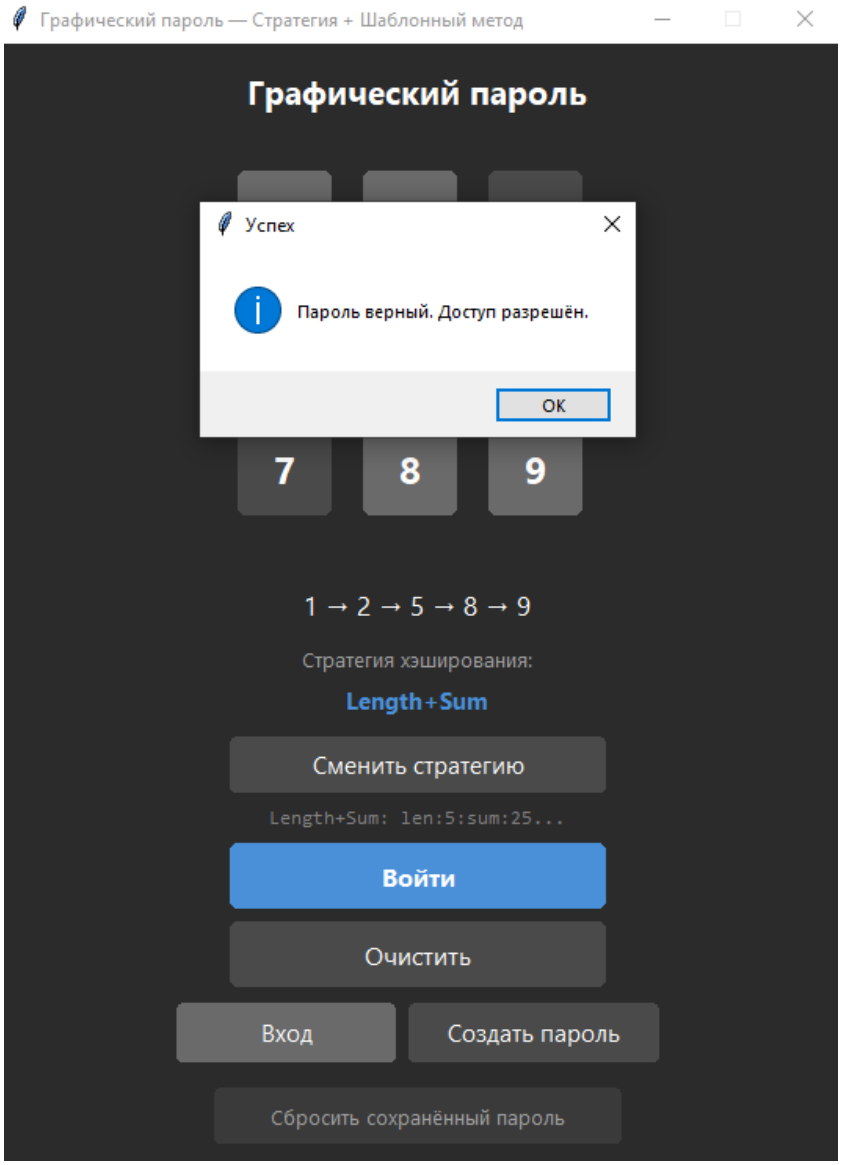


Рисунок 6 - Успешный вход (стратегия та же, что при создании)

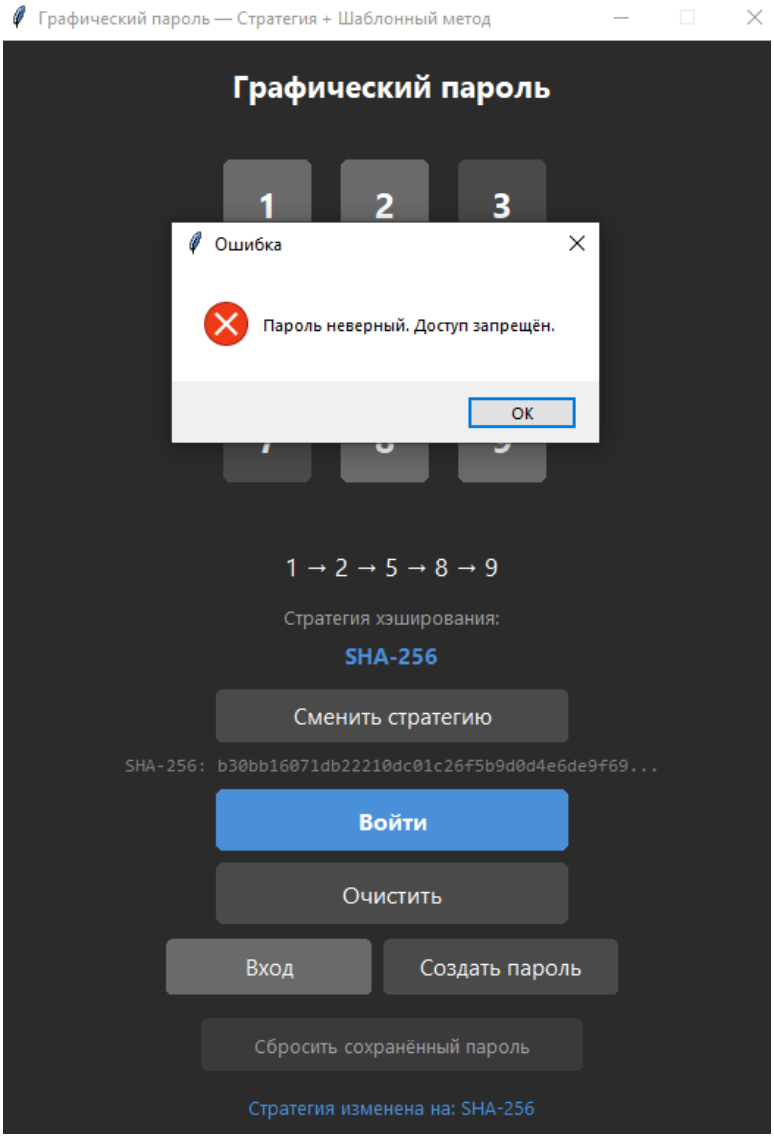


Рисунок 7 - Ошибка входа при смене стратегии (хэши не совпадают)

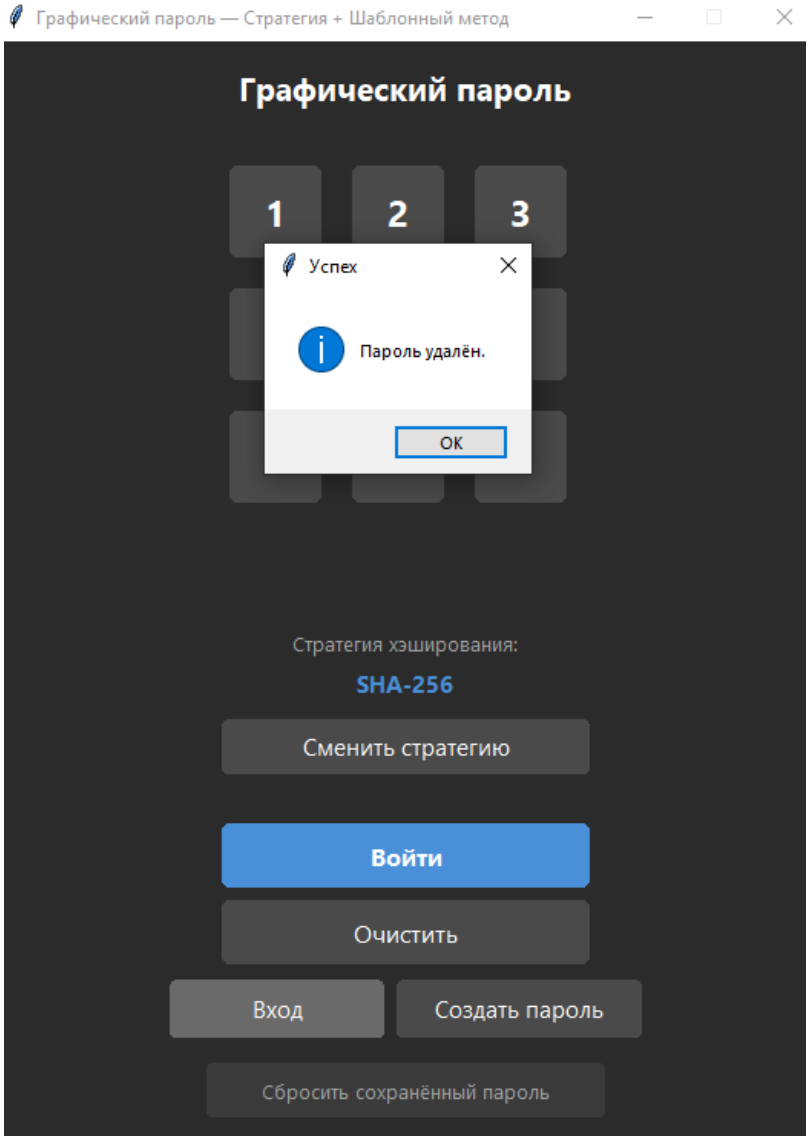


Рисунок 8 - Сброс сохранённого пароля

8. Исходный код

Основные элементы программы:

- HashingStrategy, SHA256Strategy, MD5Strategy, LengthStrategy - стратегии;
- PasswordOperation, CreatePasswordOperation, VerifyPasswordOperation - шаблонный метод;
- PasswordManager - Singleton с текущей стратегией;
- PasswordResult - результат операции;
- AppWindow - основной класс интерфейса;
- RoundedButton, rounded_rect() - скруглённые кнопки и ячейки;
- validate_sequence() - вспомогательная функция.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Были изучены и реализованы два поведенческих паттерна проектирования: «Стратегия» (Strategy) и «Шаблонный метод» (Template Method). Паттерн «Стратегия» применён для выбора алгоритма хэширования: пользователь может переключать SHA-256, MD5 и Length+Sum через HashingStrategy. Паттерн «Шаблонный метод» применён для операций над паролем: класс PasswordOperation задаёт единый порядок шагов (валидация → подготовка → хэширование → сохранение → уведомление), а CreatePasswordOperation и VerifyPasswordOperation переопределяют только конкретные шаги. В результате получена работоспособная программа, демонстрирующая практическое применение поведенческих паттернов при разработке прикладного программного обеспечения.